IS IT RL OR DATA? A CONTROLLED COMPARISON OF GRPO AND REJECTION-SAMPLING FINE-TUNING

Lakshya Chaudhry
Purdue University
lchaudhr@purdue.edu

ABSTRACT

Bauer et al. (2026) showed that in low-data reinforcement learning with verifiable rewards, training GRPO on mixed-difficulty prompts gives up to $5\times$ the sample efficiency of easy-only prompts. They did not test whether this benefit is specific to GRPO, or whether another post-training method given the same data would obtain it too. We test this on their procedural counting task with Qwen3-4B-Base and LoRA, comparing GRPO to rejection-sampling fine-tuning (RFT) on the same prompts, verifier and token cap. Six primary arms cross the two methods with three prompt sets (Easy, Mixed, and a Curated subset of prompts the base model sometimes solves, which makes GRPO's implicit filter explicit). We use three seeds per arm. Hypotheses and falsifiers were registered before training. To rule out a poorly tuned baseline, RFT receives a 27-run hyperparameter sweep, while GRPO uses Bauer et al.'s recipe unchanged. GRPO outperforms RFT at every prompt set: by 10.9 points on held-out test problems with curated prompts, and, in post hoc comparisons with matched rollout budgets, by 8.4 and 9.2 points on the easy and mixed prompts. On test the advantage survives the least favorable treatment of truncated completions; out of distribution, truncation prevents a conclusion. Curating prompts does not help RFT (-2.7 points). Mixed-difficulty data helps RFT (+3.7 points), while for GRPO the evidence is inconclusive. A secondary iterated-RFT arm, which resamples from the current policy, recovers 3.2 of the 10.9-point gap (about 30%). These are controlled measurements on one synthetic task family and one base model with three seeds per arm.

1 INTRODUCTION

Reinforcement learning with verifiable rewards, most often through GRPO (Shao et al., 2024), has become a standard way to improve reasoning in language models, but it also changes which data the model learns from. The *verifier* is a program that checks a completion's final answer against the known correct answer and assigns a score; GRPO uses this score as its reward. Rejection-sampling fine-tuning (RFT; Yuan et al., 2023) uses the same verifier differently: it samples completions, keeps those the verifier marks correct, and fine-tunes on them. The two methods are hard to compare because GRPO's objective is also a data filter: a prompt whose completions are all right or all wrong gives GRPO no learning signal, so it effectively trains on prompts of intermediate difficulty. Prior work finds that GRPO learns more efficiently from mixed-difficulty data (Bauer et al., 2026), and attributes RL's advantage over supervised fine-tuning to this implicit filtering (Lu et al., 2026). If that is right, the advantage should be recoverable by giving a supervised method the same data. We ask: with the same prompts and the same verifier, is GRPO's advantage over other post-training methods, like RFT, unique to the RL objective, or can RFT recover it from the same data? Related studies find that filtered fine-tuning on samples from the current policy performs close to GRPO (Xiong et al., 2025; Samineni et al., 2025); we test whether data sampled once from the base model is enough (§4).

We study this in the low-data setting of Bauer et al. (2026), who train Qwen3-4B with GRPO on a procedural counting task, where mixed-difficulty data gave up to $5\times$ the sample efficiency of easy-only data. The task has one integer answer per problem, so every completion can be verified

exactly. We inherit their generator, model family, LoRA configuration and GRPO recipe, and add what the comparison needs: an RFT baseline trained on the same 100-prompt sets, matched rollout budgets (19,200 sampled completions per arm on the easy and mixed sets), three seeds per arm, and hypotheses and falsifiers registered before any training run.

Our design crosses three prompt sets (Easy, Mixed, and a Curated subset of Mixed restricted to prompts the base model sometimes solves) with two training signals (RFT and GRPO), giving six arms with three seeds each, plus two single-seed GRPO controls with random and format-only rewards (§2, Table 1). Every arm uses the same scoring rule and the same 4,352-token cap, and a completion that reaches the cap scores zero. To make sure any GRPO advantage is not an artifact of a poorly tuned baseline, RFT receives a 27-run hyperparameter sweep while GRPO runs a single recipe. A secondary arm, iterated RFT, tests how much of the gap comes from sampling from the current policy.

Our findings are:

- **GRPO outperforms tuned RFT at every prompt set.** On the curated prompts GRPO leads by 10.9 pp on test. In two comparisons added after the test results were seen, it also leads on the easy and mixed prompts, where the rollout budgets are matched (8.4 and 9.2 pp). On test the advantage survives the least favorable treatment of truncated completions; out of distribution, truncation is too frequent to draw conclusions.
- **Curating prompts does not help RFT.** RFT trained on the curated prompts scores 2.7 pp below RFT on the full mixed set, contrary to our pre-registered prediction.
- **Mixed-difficulty data helps RFT; for GRPO the evidence is inconclusive.** Mixed data improves RFT by 3.7 pp; for GRPO the gain is similar in size but falls short of our criterion.
- **Iterated RFT accounts for about 30% of the gap.** Three rounds of sampling from the current policy recover 3.2 of GRPO's 10.9 pp advantage on the curated prompts.

These are controlled measurements on one synthetic task family and one base model, trained with LoRA and three seeds per arm; they are not a scaling law for RL with verifiable rewards (§5).

2 METHODOLOGY

2.1 TASK AND DATA

We use the procedural counting task of Bauer et al. (2026). A problem gives an integer range, applies 1–4 filters and 0–3 transformations, and then applies a final operation such as count, sum, or median. The answer is one integer, computed by running the pipeline. We tighten this generator in two ways: every step must change the set, and generation is stratified by range size and step count (Appendix B).

Pools. Training and in-distribution evaluation draw from a pool of 6,000 problems with ranges of span 10–200 and 2–5 steps. A separate out-of-distribution set, `ood_hard_200`, uses ranges of span 201–1,000 and 6–8 steps, which no training, validation, or selection run ever sees.

Difficulty tiers. Bauer et al. (2026) tier problems by how many frontier models solve them. We tier by the model we train. We sample 8 completions per problem from the base model at $T{=}1.0$ and label a problem *easy* if $\geq 6/8$ are correct, *medium* if 2–5 are, and *hard* if ≤ 1 is. Unlike an external panel, the trained model's own pass rate measures difficulty where it matters for learning: whether a group of $G{=}8$ GRPO samples contains both successes and failures depends on the policy's success rate. The same eight samples also define `train_curated`, so difficulty and informativeness are measured on one scale.

Data splits. The splits are disjoint by problem and by pipeline structure:

- `train_easy_100`: 100 easy problems.
- `train_mixed_100`: 33 easy / 33 medium / 34 hard.

- `train_curated`: the 73 problems in `train_mixed_100` with between 1 and 7 correct of 8 (26 easy / 33 medium / 14 hard). These are the prompts that would give a GRPO group a learning signal. The set is fixed once and never re-selected.
- `val_mixed_100`: 33 / 33 / 34, the only split used for selection.
- `test_300`: 100 per tier.

We also report GSM8K (the first 500 test problems) as a secondary transfer set.

Prompt, scoring, and token cap. All base-model runs use one plain-text prompt, with no chat template, as is usual for RL from a base model (Guo et al., 2025): it asks the model to reason step by step and end with a line `Answer: <integer>`. The reasoning is ordinary text before that line and no `<think>` tags are used. Scoring checks the number, not the format:

- The answer is the last integer on an answer line, else in a `\boxed{}`, else anywhere in the completion.
- Correctness is exact match.
- The reward for every primary arm is this binary correctness, with no format term.

Every run uses a cap of 4,352 completion tokens, for both training and evaluation. The cap is derived from base-model completion lengths, with Bauer et al.'s 2,048 as a floor (Appendix D). **A completion that reaches the cap scores zero**, in GRPO rewards, in RFT filtering, and at evaluation.

2.2 Arms

We cross the prompt set (Easy, Mixed, Curated) with the training signal (RFT or GRPO), which gives six arms with three seeds each (Table 1). All arms train a LoRA adapter (rank 64, α=16, all linear layers) on Qwen3-4B-Base with AdamW, a cosine schedule with 10% warmup, and gradient clipping at 1.0. Every run is evaluated at its last step.

RFT. Rejection-sampling fine-tuning (Yuan et al., 2023) samples completions from a model, keeps those a verifier marks correct, and fine-tunes on them. We sample 192 completions per prompt from the base model once, at T=1.0, and all RFT arms and seeds share this draw. Each arm keeps the correct completions for its prompts, removes exact duplicates, and fine-tunes on them with a completion-only loss. No other data is mixed in. Because the kept set follows the base model's pass rate, it is dominated by easy prompts: in the RFT Mixed split, only 595 of 7,961 kept samples come from hard prompts.

GRPO. Group Relative Policy Optimization (Shao et al., 2024) is a policy-gradient method that replaces a learned value function with a group baseline: each prompt is sampled several times, and each completion's advantage is its reward normalized by the group's mean and standard deviation. We use the `GRPOTrainer` of TRL (von Werra et al., 2020) (v1.13.0) for 300 steps. Each step takes 8 prompts, with 8 completions per prompt, at a learning rate of 5×10^{-5}. The KL coefficient is 0, and there is one update per sampled batch. The loss is normalized at the token level (`loss_type=dapo`; Yu et al., 2025). Groups whose 8 rewards are all equal contribute no gradient, which is the implicit filtering that `train_curated` makes explicit. Capped completions stay in the loss with reward 0. Rollouts come from vLLM (Kwon et al., 2023) with the current adapter at every step.

Iterated RFT. Iterating RFT, where each round samples from the latest policy, is the approach of ReST and ReST^{EM} (Gulcehre et al., 2023; Singh et al., 2024). This arm runs three rounds of RFT on the 73 curated prompts, with 64 samples per prompt per round. Round 1 is taken from the base draw, and rounds 2–3 are sampled from the current policy. Each round continues training the previous adapter. The arm refreshes the sampling policy 3 times, against GRPO's 300, and keeps RFT's correct-only objective. It is a secondary arm (§2.4).

Controls. The controls run the GRPO recipe on `train_mixed_100`, one seed each:

Table 1: Arms and budgets (means per seed). Rollouts: sampled completions available / used for training. RFT-Mixed has a correct sample for only 95 of its 100 prompts.

Arm	Seeds	Prompts	Rollouts (avail. / used)	Train tokens	Opt. steps
RFT-Easy	3	100	19,200 / 13,834	47.2M	6,920
RFT-Mixed	3	100	19,200 / 7,961	15.9M	1,992
RFT-Curated	3	73	14,016 / 6,653	13.4M	1,664
GRPO-Easy	3	100	19,200 / 19,200	12.4M	300
GRPO-Mixed	3	100	19,200 / 19,200	23.0M	300
GRPO-Curated	3	73	19,200 / 19,200	20.6M	300
IterRFT-Curated (secondary)	3	73	14,016 / 7,828	16.7M	1,963
C1 random reward	1	100	19,200 / 19,200	9.3M	300
C2 format reward	1	100	19,200 / 19,200	10.8M	300

- **C1** uses a random Bernoulli(0.5) reward.
- **C2** rewards any uncapped completion that states an explicit final answer, whether or not it is correct.

2.3 Budgets and tuning

Budgets. Each RFT/GRPO pair trains on the same prompts, and the Easy and Mixed pairs are also matched on rollouts (Table 1). GRPO samples 19,200 rollouts in total (300 steps $\times$ 8 prompts per step $\times$ 8 completions per prompt), and RFT draws the same 19,200 up front as 192 completions for each of the 100 prompts. On the curated set, RFT has the 14,016 samples that belong to its 73 prompts (73×192), while GRPO still samples 19,200. Our central comparison, the Curated pair, is therefore matched on prompts but not on rollouts. We do not equalize training tokens or optimizer steps: RFT trains for several epochs on correct samples only, while GRPO trains once on every sample, correct or not. That asymmetry is deliberate. It is the mechanism we compare: GRPO also learns from failed samples and from how each sample ranks within its group, while RFT sees only successes.

Tuning. We tune RFT and not GRPO, so that any advantage for GRPO cannot be explained by a poorly tuned baseline. Each RFT arm gets a 3×3 sweep over learning rate (10^{-5}, 5×10^{-5}, 10^{-4}) and epochs (2, 4, 8), 27 runs in total, and we keep the configuration with the best validation accuracy. GRPO runs a single recipe with no search, taken from Bauer et al. (2026): LoRA configuration, learning rate, schedule, 300 steps, and 8 completions per prompt at $T{=}1.0$. We depart from that recipe in two places. Our reward is binary correctness, without their format bonus and step-count penalty, and we use 8 prompts per step (64 completions); Bauer et al. (2026) report an effective batch size of 8 without stating whether it counts prompts or completions. Iterated RFT reuses RFT-Curated's configuration. Appendix E lists all values.

2.4 Evaluation

Metrics. The primary metric is greedy accuracy on `test_300` and `ood_hard_200`, generated with vLLM under the same prompt, scoring rule, and cap for every model. We also report:

- mean@8 accuracy at $T{=}1.0$;
- pass@k for $k \leq 64$ on the first 100 test problems;
- per-tier accuracy;
- GSM8K accuracy;
- truncation rate for every number.

Three reference models are evaluated the same way: Qwen3-4B (non-thinking), Qwen2.5-7B-Instruct, and Llama-3.1-8B-Instruct.

Statistics. For each arm we report the mean ± standard deviation over seeds, with every seed shown. Each run also gets a 95% bootstrap CI over problems. Arm contrasts are paired by seed, with a paired bootstrap CI. A contrast counts as an effect if both of these hold:

1. it exceeds twice the pooled seed standard deviation on `test_300`;
2. it has the same sign on `ood_hard_200`.

Runs with more than 5% truncation on `test_300` are flagged.

Pre-registration. The three comparisons (Mixed vs. Easy within each method, RFT-Curated vs. RFT-Mixed, and GRPO-Curated vs. RFT-Curated), our predictions for each, and the success criterion above were committed before any training run (Appendix A). The iterated-RFT arm, the GRPO–RFT comparisons on the easy and mixed prompts, and two truncation analyses were added after the test results were seen and are marked *secondary* or *post hoc* wherever they appear.

3 RESULTS

Table 2 reports greedy accuracy and truncation for every arm, and Table 3 reports every paired comparison against the success criterion. We organize the results by comparison, starting with the training signal.

3.1 GRPO OUTPERFORMS TUNED RFT AT EVERY PROMPT SET

When we compare GRPO and RFT on the curated prompts, GRPO is 10.9 pp more accurate on `test_300` (77.8% vs. 66.9%; Table 3). All three seed pairs agree (+12.0, +10.3, +10.3 pp), and the gap is almost four times the 2σ threshold. On `ood_hard_200` the gap has the same sign but is smaller, and its interval includes zero (+3.0 pp, $[-2.5, +8.5]$). It meets the success criterion.

Two post hoc comparisons, on the easy and mixed prompts, where the methods are also matched on rollouts, give the same ordering. On the easy prompts GRPO is 8.4 pp more accurate on test and 8.3 pp on OOD; neither arm exceeds the truncation flag, and both intervals exclude zero. On the mixed prompts GRPO is 9.2 pp more accurate on test and 3.5 pp on OOD. GRPO's advantage over RFT is therefore of similar size (8–11 pp on test) whether or not GRPO samples more rollouts than RFT, so the extra rollouts GRPO receives on the curated set cannot account for its advantage there.

RFT changes the model but not its greedy accuracy. Single-round RFT barely moves greedy test accuracy relative to the base model (Table 2): RFT-Easy is 1.4 pp below it, and the other two arms have intervals that include zero. Relative to Base:

- test truncation falls from 4.3% to 1.7–3.3%, because RFT learned to stop;
- sampled accuracy (mean@8, T=1.0) rises from 42.5% to 51.2–54.4%;
- the share of sampled completions with an explicit answer line rises from 77% to 82–85%.

The format-only control raises mean@8 even further (§3.1, Controls), so the sampled gain cannot be separated from formatting. On greedy accuracy, which is our primary metric, single-round RFT with the best of nine configurations does not improve on the base model at this budget. GRPO does.

Where the gain lands. Most of GRPO's gain over Base comes from the hard tier: 85% of the Mixed arm's gain, 76% of the Curated arm's, and 56% of the Easy arm's. On the 100 hard test problems:

- Base scores 29.0%;
- the RFT arms score 23.3–29.7%;
- the GRPO arms score 40.7% (Easy), 58.0% (Mixed) and 52.7% (Curated).

The hard tier is also where GRPO truncates most (13% of hard problems for the Mixed and Curated arms), so these GRPO numbers are lower bounds. On the easy tier, where Base already scores 92.0%, all six primary arms stay between 92% and 97% (Figure 1).

Table 2: Greedy accuracy (%), mean ± sd over 3 seeds, with the share of completions cut at the 4,352-token cap (trunc.). ‡All three seeds exceed 5% truncation on `test_300`. Base, controls and the reference model are single runs.

	`test_300`		`ood_hard_200`	
Arm	acc.	trunc.	acc.	trunc.
Base	67.3	4.3	25.0	13.0
RFT-Easy	65.9 ± 1.1	2.4	25.5 ± 0.5	10.5
RFT-Mixed	69.6 ± 1.0	2.0	27.3 ± 1.2	12.2
RFT-Curated	66.9 ± 1.1	3.1	28.5 ± 1.8	14.8
GRPO-Easy	74.3 ± 3.9	3.0	33.8 ± 3.1	26.2
GRPO-Mixed	78.8 ± 0.5	8.7‡	30.8 ± 2.3	47.5
GRPO-Curated	77.8 ± 1.7	7.6‡	31.5 ± 3.9	34.8
IterRFT-Curated (secondary)	70.1 ± 0.5	1.6	28.8 ± 2.4	15.0
C1 random reward	47.7	4.7	22.0	7.5
C2 format reward	63.7	1.0	27.0	10.5
Qwen3-4B (reference)	74.7	4.0	45.5	22.5

Table 3: Paired comparisons (A − B, pp), mean over seeds with a paired bootstrap 95% CI. 2σ: twice the pooled seed sd. ✓: meets the success criterion ($|\Delta_{\text{test}}| > 2\sigma$ and the same sign on OOD). The easy and mixed GRPO − RFT rows and the iterated-RFT rows are post hoc.

Comparison (A − B)	Δ `test_300`	2σ	Δ `ood_hard_200`	Crit.
Training signal: GRPO − RFT on the same prompts				
GRPO-Curated − RFT-Curated	+10.9 [+6.7, +15.1]	2.8	+3.0 [−2.5, +8.5]	✓
GRPO-Easy − RFT-Easy	+8.4 [+4.8, +12.1]	5.8	+8.3 [+3.7, +13.0]	✓
GRPO-Mixed − RFT-Mixed	+9.2 [+4.8, +13.7]	1.6	+3.5 [−2.0, +9.0]	✓
Prompt curation				
RFT-Curated − RFT-Mixed	−2.7 [−5.2, 0.0]	2.1	+1.2 [−2.5, +5.3]	–
GRPO-Curated − GRPO-Mixed	−1.0 [−3.7, +1.7]	2.5	+0.7 [−2.7, +4.0]	–
Mixed-difficulty data				
RFT-Mixed − RFT-Easy	+3.7 [+0.2, +7.0]	2.1	+1.8 [−2.0, +5.7]	✓
GRPO-Mixed − GRPO-Easy	+4.4 [+1.1, +7.9]	5.6	−3.0 [−7.3, +1.2]	–
Iterated RFT				
IterRFT-Curated − RFT-Curated	+3.2 [+0.2, +6.2]	1.7	+0.3 [−3.5, +3.8]	✓
GRPO-Curated − IterRFT-Curated	+7.7 [+3.7, +11.7]	2.5	+2.7 [−2.2, +7.3]	✓

Reliability, not coverage. GRPO makes a single attempt far more likely to succeed: per-sample accuracy (mean@8) rises from 42.5% for Base to 70.1–77.1%. It does not enlarge the set of problems the model can ever solve. Given 64 attempts on the first 100 test problems, Base already solves 98.0%, and the GRPO arms solve 95.7–96.7%, a difference within the uncertainty of a 100-problem sample. GRPO therefore turns problems the base model solves occasionally into problems it solves reliably, consistent with prior reports that RL with verifiable rewards sharpens the base model's distribution rather than expanding it (Yue et al., 2025) (Appendix H).

Truncation. GRPO-trained models write longer reasoning traces, so more of their completions hit the 4,352-token cap and are scored wrong (1.3–10.3% per seed on test, against at most 3.3% for RFT; Tables 2 and 8). On test the advantage holds even under the least favorable assumption, crediting every truncated RFT completion as correct and every truncated GRPO completion as wrong (+6.0 to +7.8 pp across the three prompt sets). On OOD, where 20–55% of GRPO completions are cut, truncation is large enough to change the differences, so we draw no OOD conclusions from them (Appendix G).

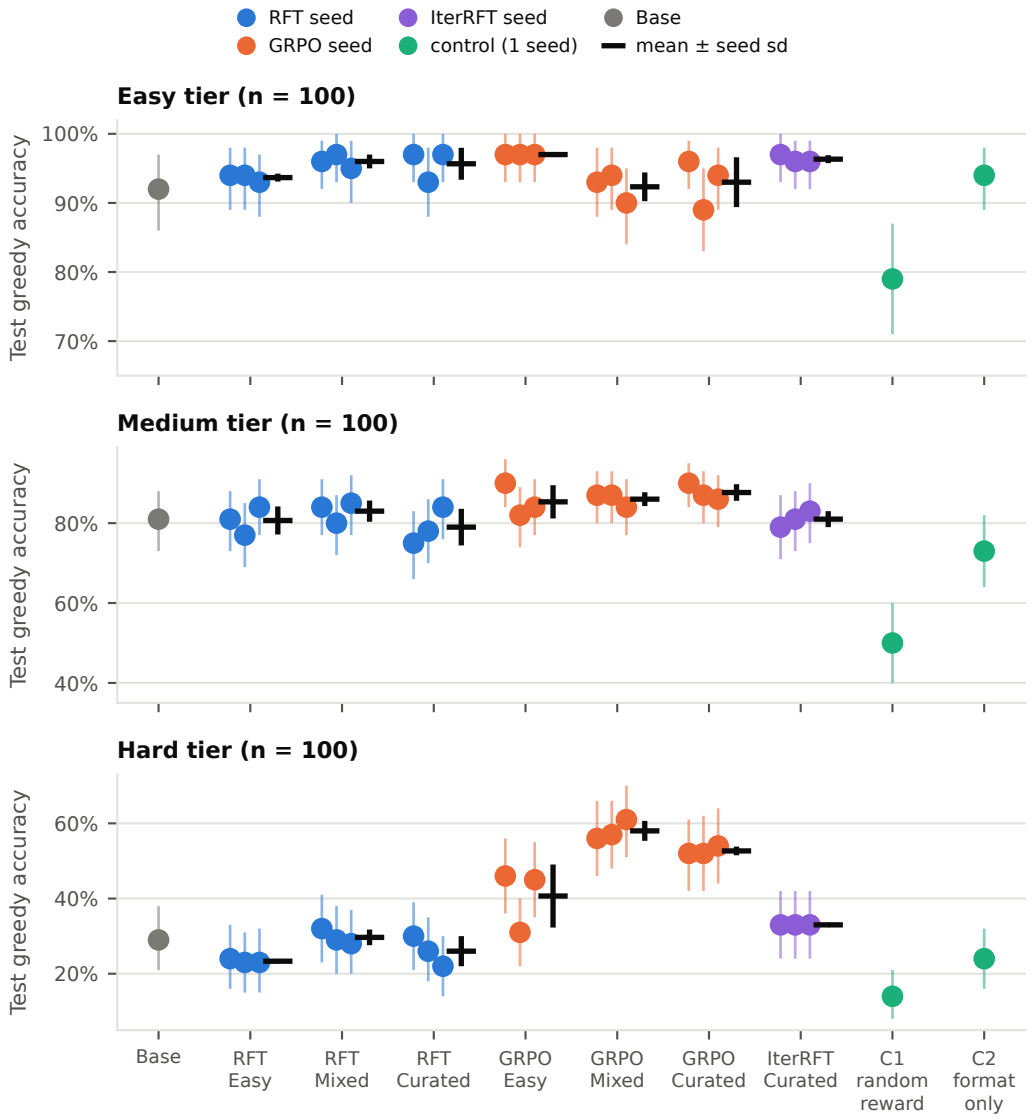


Figure 1: Greedy accuracy on `test_300` by difficulty tier (100 problems per tier). Dots are seeds with 95% bootstrap CIs; black bars are the mean ± sd over seeds. Controls and Base are single runs. Each row has its own y-axis range. RFT and GRPO are close on easy problems and separate on hard ones.

Controls. The gain is not reproduced by rewards that ignore correctness. The two controls share GRPO-Mixed's prompts, recipe and seed 1, and differ only in the reward assigned. On test, the correctness reward reaches 78.7%, the format-only reward 63.7%, and the random reward 47.7%, against 67.3% for Base. On OOD the three score 29.5%, 27.0% and 22.0%, against 25.0% for Base. Each control is a single seed, so these numbers are descriptive.

The format-only reward does raise mean@8, from 42.5% to 60.3%, and pushes the answer-line rate to 99%. Therefore it is plausible that any sampled-accuracy gain can come from formatting alone.

3.2 CURATING PROMPTS DOES NOT HELP RFT

Restricting RFT to the prompts with mixed outcomes, the ones that give GRPO a learning signal, does not improve it:

- RFT-Curated scores 2.7 pp *below* RFT-Mixed on test $[-5.2, 0.0]$, with all three seed pairs negative;
- on OOD it scores 1.2 pp above.

We had initially predicted that curation would recover most of GRPO's advantage over RFT-Mixed (8.2 pp). Instead, the recovered share is negative (-0.32 as a point estimate), and the prediction is not supported.

The two arms also differ in the learning rate selected on validation (10^{-5} vs. 5×10^{-5}). And a 2.7 pp gap is about the size of the changes we see when the same run is re-evaluated on a different machine (up to 2.3 pp on test; §5). We conclude that curation does not help RFT, but we do not conclude that it hurts.

For GRPO, explicit curation changes nothing detectable: GRPO-Curated is within 1.0 pp of GRPO-Mixed on test. This matches the implicit-filtering account. GRPO already gets no gradient from groups where every completion agrees, and on the mixed prompts the share of such groups rises from 26–29% in the first 25 steps to 61–70% in the last 25 (Figure 2). Over training, GRPO filters its own prompts, so filtering them in advance adds little.

3.3 MIXED-DIFFICULTY DATA HELPS RFT; FOR GRPO THE EVIDENCE IS INCONCLUSIVE

- **RFT:** training on the mixed set beats the easy set by 3.7 pp on test $[+0.2, +7.0]$, with the same sign on OOD (+1.8 pp). This meets the criterion.
- **GRPO:** the test difference is similar (+4.4 pp) but below the threshold, because GRPO-Easy's seeds vary widely (70.0–77.7%). The OOD sign reverses (-3.0 pp), and every GRPO-Mixed seed is truncation-flagged.

The effect Bauer et al. (2026) report for RL therefore also appears for SFT on correct samples in our setting. For RL itself, we cannot confirm it, due to the truncation we see for GRPO on OOD.

Two caveats apply to the RFT comparison:

- The two arms were selected at different epoch counts (8 vs. 4), so the data effect also carries an optimization difference.
- The 3.7 pp gap is only somewhat larger than the cross-machine variation. Because the RFT seeds share one base-model draw, their seed spread, and hence the 2σ threshold, omits sampling variance.

3.4 ITERATED RFT ACCOUNTS FOR ABOUT 30% OF THE GAP

Sampling from the improving policy recovers some of GRPO's advantage, but not most of it.

What iterated RFT tests. One difference between the methods is where the training data comes from. Single-round RFT learns only from the base model's samples, while GRPO samples from its current policy at every step. Iterated RFT narrows that difference: it runs three rounds of RFT on the curated prompts, and each round samples from the model trained in the previous round. It still learns only from correct samples.

Results. Iterated RFT improves on single-round RFT by 3.2 pp on test $[+0.2, +6.2]$, with no gain on OOD (+0.3 pp). GRPO still leads it by 7.7 pp on test $[+3.7, +11.7]$.

How much of the gap it explains. These two steps add up exactly to GRPO's advantage over single-round RFT on the curated prompts ($3.2 + 7.7 = 10.9$ pp). As a descriptive split, iterated RFT accounts for about 30% of GRPO's advantage, and about 70% remains. The remaining 70%

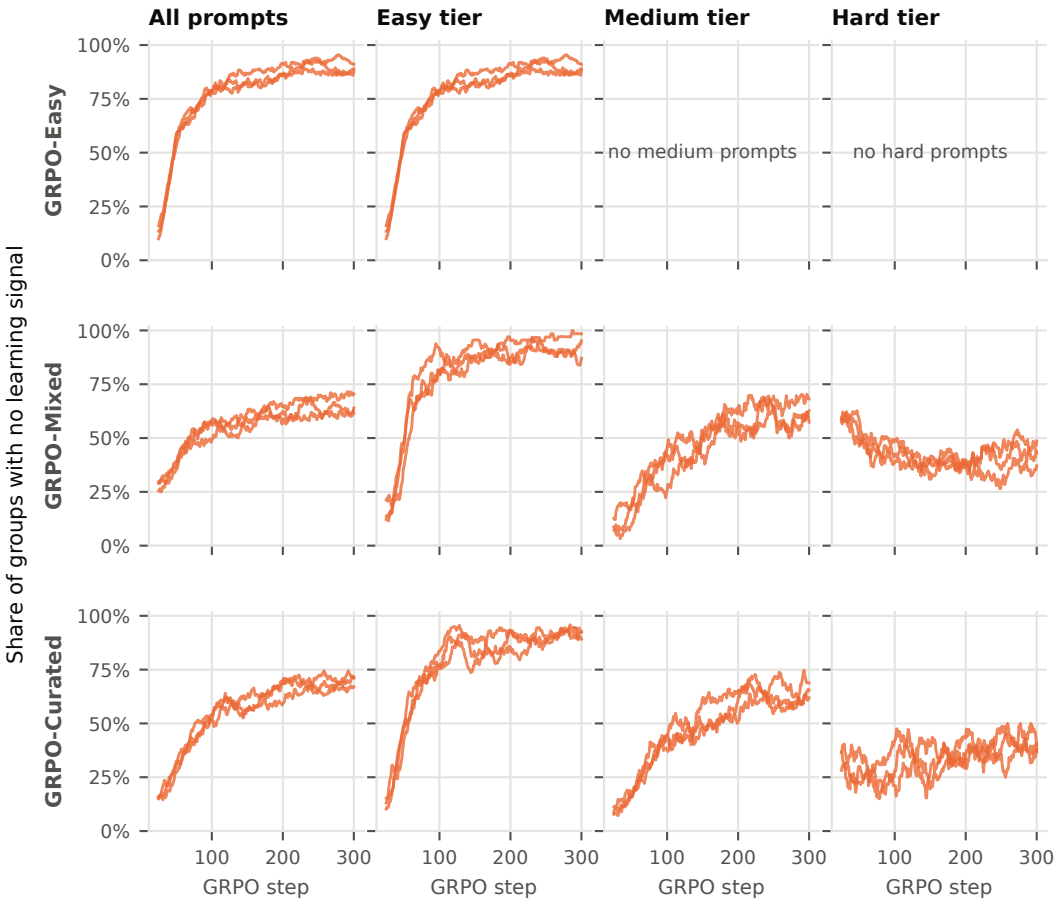


Figure 2: Share of GRPO groups whose eight completions all received the same reward, and so contribute no gradient, over training. One line per seed, 25-step trailing mean. Columns split the prompts by difficulty tier; the easy prompt set contains only easy prompts.

reflects everything GRPO does that iterated RFT does not: learning from failed samples, comparing samples within a group, refreshing its samples 300 times rather than 3, a larger learning rate, and more rollouts (19,200 vs. 14,016).

Two caveats limit the 30%. First, it is not purely an effect of fresher samples: iterated RFT also takes more optimizer steps (1,963 vs. 1,664) and trains on more samples (7,828 vs. 6,653). Second, the 3.2 pp gain is close to the change we see when the same model is re-evaluated on a different machine (up to 2.3 pp).

4 Related work

Fine-tuning on verified samples. A line of work improves a model by fine-tuning it on those of its own outputs that pass a check. STaR (Zelikman et al., 2022) does this in a loop for rationales that lead to a correct answer. RFT (Yuan et al., 2023) collects correct reasoning paths sampled from fine-tuned models and adds them to the fine-tuning data. ReST (Gulcehre et al., 2023) alternates between sampling from the current policy and improving the policy on those samples with offline RL, and ReST^{EM} (Singh et al., 2024) repeats a loop of sampling, filtering on binary feedback and fine-tuning a few times. Our RFT arm is a single-round form that samples from the base model and trains on the verified samples alone, and our iterated-RFT arm is the repeated form.

Is RL more than filtered supervised learning? Several recent studies compare GRPO with simpler methods that train on filtered samples. Xiong et al. (2025) find that RAFT, which keeps only positively rewarded samples, is competitive with GRPO and PPO, and attribute GRPO's main advantage to discarding prompts whose responses are all incorrect. Samineni et al. (2025) argue that, under the usual modeling assumptions, the GRPO objective reduces to a filtered, iterative form of supervised fine-tuning, and report that this form, using both positive and negative samples, performs comparably to GRPO. Zhu et al. (2025) split the RL signal into reinforcing correct samples and penalizing incorrect ones. They find that reinforcing correct samples alone raises pass@1 but lowers pass@k at larger k, while penalizing incorrect samples alone often matches GRPO.

Our results differ from the first two: we find a gap of 8–11 pp between GRPO and RFT. The settings differ in ways that could account for this. Their filtered methods sample from the current policy at every iteration, and Xiong et al. (2025) draw prompts from about 860k problems. Our RFT samples once from the base model and trains for several epochs on at most 100 prompts. Our iterated-RFT arm, which refreshes its samples three times, recovers part of the gap (§3.4). We did not test an RFT that refreshes at every step, so our results do not contradict theirs. What they show is narrower: data sampled once from the base model was not enough in our setting. Our pass@k results are consistent with Zhu et al. (2025) and Yue et al. (2025): all six primary arms raise per-sample accuracy (mean@8), and none raises pass@64 above the base model (Appendix H).

Difficulty and data selection. Bauer et al. (2026) show that GRPO learns more efficiently from mixed-difficulty prompts in low-data settings. Lu et al. (2026) explain RL's generalization advantage in vision-language models by an implicit filter that favors prompts with mixed outcomes, and show that supervised fine-tuning on a difficulty-filtered training set can surpass GRPO out of distribution. DAPO makes a similar filter an explicit part of RL training, by over-sampling and then dropping prompts whose completions are all correct or all incorrect (Yu et al., 2025). Our `train_curated` set matches the medium-difficulty definition of Lu et al. (2026), a mix of correct and incorrect responses among eight samples from the initial model, and corresponds to their medium-only variant. Their strongest variant also keeps the easy samples, which we did not test as a separate arm. Our setting differs from theirs in two ways. They fine-tune on ground-truth labels for image classification and visual grounding, while RFT fine-tunes on the model's own sampled reasoning. And in RFT the number of training samples per prompt still follows the base model's pass rate. In our setting the filter did not help (§3.2).

Response length. Longer outputs under GRPO have been reported before. Liu et al. (2025) identify a bias in the GRPO objective that increases response length, especially for incorrect outputs, and Samineni et al. (2025) argue that the standard formulation indirectly incentivizes longer outputs. Our GRPO arms, which use the token-level loss normalization of DAPO (Yu et al., 2025), also write longer completions than the base model and reach the token cap more often (Appendix G).

5 LIMITATIONS

This is a controlled comparison on one synthetic task family, not a general result about RL with verifiable rewards.

Task and model. All training data comes from the procedural counting generator of Bauer et al. (2026), with our two constraints, and we train one base model (Qwen3-4B-Base) with LoRA only. We do not test other task families, model sizes, or full fine-tuning. Bauer et al. (2026) study three task families (counting, graph reasoning and spatial reasoning), and we use only counting. With a fixed compute budget (252.7 GPU-hours; Appendix J) we chose three seeds, a tuned baseline and two controls on one task over a second task or model.

Scale of evidence. Primary arms have three seeds, and the two controls and Base one each. Our success criterion compares a difference to seed variation and checks its sign out of distribution; it is not a formal significance test. The three RFT seeds share one base-model draw, so their spread omits sampling variance. Re-evaluating a model on a different machine changes test accuracy by up to 2.3 pp. The smaller effects (curation, mixed data for RFT, iterated RFT) are close to that, so we treat them as suggestive.

Budgets and tuning. The Easy and Mixed pairs match prompts and rollouts; the Curated pair matches prompts only (19,200 vs. 14,016 rollouts). We do not match training tokens or optimizer steps. RFT received a 27-run hyperparameter sweep selected on 100 validation problems, while GRPO used a single recipe from Bauer et al. (2026). This favors RFT, but RFT's selected configurations also differ within its own data comparisons (epochs for mixed data, learning rate for curation).

The RFT baseline. Three features of our RFT may understate what filtered fine-tuning can do. First, it keeps every correct sample with no per-prompt cap, so its training set follows the base model's pass rate and is dominated by easy prompts (595 of the 7,961 samples in RFT-Mixed come from hard prompts), while GRPO samples eight completions for every prompt it draws. A per-prompt cap or tier-balanced sampling might narrow the gap, since GRPO's gain is concentrated on the hard tier (§3.1). Second, it samples once from the base model, whereas the filtered methods reported to match GRPO resample from the current policy at every iteration (Xiong et al., 2025; Samineni et al., 2025); our iterated arm refreshes only three times. Third, single-round RFT did not improve greedy accuracy over the base model, so the gap between GRPO and RFT is about as large as GRPO's gain over the base model. Our results therefore show that GRPO outperforms this RFT recipe at this budget, not that it outperforms filtered fine-tuning in general. Testing these variants was beyond our compute budget.

What the gap does not isolate. GRPO's advantage bundles several differences: learning from failed samples, group-relative reward, 300 policy refreshes rather than one or three, a larger learning rate, and more rollouts on the curated set. Iterated RFT separates out part of the refresh effect; we do not separate the rest.

Truncation and scoring. A completion that reaches the 4,352-token cap scores zero, and GRPO reaches it more often. On test, the GRPO advantage survives crediting every truncated completion against it. On OOD, 20–55% of GRPO completions are cut, so we draw no OOD conclusions. The extractor falls back to the last integer when no answer line or `\boxed{}` is present; this applies to every arm alike.

Analyses added after seeing test results. The GRPO–RFT comparisons on the easy and mixed prompts, iterated RFT, and the truncation analyses were defined after the primary test results were seen, and are outside the pre-registered comparisons (Appendix A).

6 Conclusion

We compared GRPO with rejection-sampling fine-tuning (RFT) on the procedural counting task of Bauer et al. (2026), training Qwen3-4B-Base with LoRA. Within each comparison both methods saw the same prompts and the same verifier, and on the easy and mixed prompt sets the same number of rollouts. Our hypotheses, predictions and success criterion were committed before any training run.

GRPO outperformed RFT at every prompt set. On the curated prompts it led by 10.9 pp on test, and on the easy and mixed prompts, in comparisons added after the test results were seen, it led by 8.4 and 9.2 pp. On test, this advantage survives the least favorable treatment of truncated completions; out of distribution, truncation prevents a conclusion. Curating prompts did not help RFT, contrary to our pre-registered prediction: RFT on the curated prompts scored 2.7 pp below RFT on the full mixed set. Mixed-difficulty data helped RFT by 3.7 pp, while for GRPO the evidence was inconclusive. In a secondary arm, three rounds of iterated RFT recovered 3.2 of the 10.9 pp gap, about 30%. None of the data-side changes we gave RFT recovered most of GRPO's advantage.

These results come from one synthetic task family, one base model, LoRA training and three seeds per arm. They measure the two methods under fixed budgets, and are not a general claim about reinforcement learning with verifiable rewards.

REPRODUCIBILITY STATEMENT

Code, adapters, per-seed metrics, and the pre-registration are available in the project repository at `https://github.com/LakshyaChaudhry/rl-or-data`.

AI USE STATEMENT

The research question, experimental design, hypotheses and pre-registration are all ours, as are the core components that define the results: answer verification, RFT selection, the SFT loss, the reference GRPO implementation, and the evaluation metrics. An AI coding assistant (Claude) was used to implement the surrounding engineering to our specifications (data generation pipeline, training and evaluation harness, analysis scripts and figures) and to draft and edit the text of this paper from our outlines. We directed and reviewed all of this work, checked every reported number against the logged experiment outputs, and take full responsibility for the paper's content.

REFERENCES

Justin Bauer, Thomas Walshe, Derek Pham, Harit Vishwakarma, Armin Parchami, Frederic Sala, and Paroma Varma. Learning from less: Measuring the effectiveness of RLVR in low data and compute regimes. In *Proceedings of Machine Learning and Systems (MLSys)*, 2026.

Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (ReST) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

Aojun Lu, Tao Feng, Hangjie Yuan, Wei Li, and Yanan Sun. Why does RL generalize better than SFT? a data-centric perspective on VLM post-training. *arXiv preprint arXiv:2602.10815*, 2026.

Soumya Rani Samineni, Durgesh Kalwar, Karthik Valmeekam, Kaya Stechly, and Subbarao Kambhampati. RL in name only? analyzing the structural assumptions in RL post-training for LLMs. *arXiv preprint arXiv:2505.13697*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Avi Singh, John D. Co-Reyes, Rishabh Agarwal, et al. Beyond human data: Scaling self-training for problem-solving with language models. *Transactions on Machine Learning Research*, 2024. arXiv:2312.06585.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformers Reinforcement Learning, 2020. URL `https://github.com/huggingface/trl`.

Wei Xiong, Jiarui Yao, Yuhui Xu, Bo Pang, Lei Wang, Doyen Sahoo, Junnan Li, Nan Jiang, Tong Zhang, Caiming Xiong, and Hanze Dong. A minimalist approach to LLM reasoning: from rejection sampling to reinforce. *arXiv preprint arXiv:2504.11343*, 2025.

Qiying Yu, Zheng Zhang, Ruofei Zhu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Xinyu Zhu, Mengzhou Xia, Zhepei Wei, Wei-Lin Chen, Danqi Chen, and Yu Meng. The surprising effectiveness of negative reinforcement in LLM reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

APPENDIX

A PRE-REGISTRATION

Commitment. The hypotheses and success criterion below were committed on 2026-09-07 and the predictions on 2026-09-12, before any training run. The first training job started on 2026-09-13. The text is quoted verbatim and has not been edited to match the results. The pre-registration labels the three comparisons H1–H3; in the main text they are the mixed-data, curation, and Curated-pair comparisons.

Hypotheses.

> **H1 (data effect):** Mixed-difficulty training data improves held-out performance relative to easy-only data *for both* RFT and GRPO. Test: (RFT-Mixed − RFT-Easy) and (GRPO-Mixed − GRPO-Easy), each vs. seed spread. If both are positive, the Bauer et al. effect is broader than RLVR.
>
> **H2 (implicit filtering):** RFT trained on prompts selected by the *same current-policy criterion that makes GRPO groups informative* (1–7 correct of 8) recovers a substantial fraction of GRPO's advantage. Test: (RFT-Curated − RFT-Mixed) relative to (GRPO-Curated − RFT-Mixed). If large, RL's edge is mostly an expensive data-selection mechanism (cf. Lu et al. 2026, DC-SFT, in VLMs).
>
> **H3 (on-policy / negative-feedback effect):** GRPO-Curated outperforms RFT-Curated by more than run-to-run variability despite identical prompt and sampling budgets. Residual gain can only come from what RFT lacks: negative samples, relative reward, on-policy generation, repeated policy-dependent exploration.

Falsifiers.

> for H1: Falsified by an asymmetry: GRPO-Mixed $>$ GRPO-Easy by more than seed spread, but RFT-Mixed $\approx$ RFT-Easy (or worse). That would mean the mixed-difficulty advantage only exists through the RL objective
>
> for H2: Falsified if RFT-Curated $\approx$ RFT-Mixed – the same prompt filter that makes GRPO groups informative does nothing for SFT – while GRPO-Curated stays well above both. Also falsified if RFT-Curated is worse than RFT-Mixed
>
> for H3: Falsified if the difference on test_300 is within $2\times$ the pooled seed std, or it flips sign on ood_hard_200, or RFT-Curated $\geq$ GRPO-Curated. Any of those means: once data selection is controlled, the RL objective adds nothing you can measure at this budget.

Success criterion.

> An arm-vs-arm difference "counts" if (a) $|\Delta$ greedy accuracy on test_300$| > 2 \times$ the pooled seed std of the two arms, and (b) the sign of Δ is the same on ood_hard_200. Report all comparisons regardless.

Predictions.

> RFT-all mixed vs easy: RFT-Mixed $>$ RFT-Easy on test_300 (same direction on ood_hard_200), gap $>$ seed noise – mixed difficulty helps SFT too, so Bauer's effect is not RL-only (H1). GRPO vs RFT-curated: GRPO-Curated $>$ RFT-Curated on test_300 with the same sign on ood, but the residual is modest; most of GRPO's edge vs RFT-Mixed is recovered by curation (H2), with a leftover from negatives / relative reward / on-policy (H3).

Logged deviations.

1. **RFT trains the end-of-sequence token** (2026-09-13). The original RFT recipe did not train the stop token, and validation truncation rose to 36% during the first sweep. All sweep runs were restarted with the token trained; only validation numbers had been seen.
2. **Iterated RFT reuses RFT-Curated's hyperparameters** with no sweep of its own (2026-09-20).
3. **Exploratory re-evaluation at a larger cap** (8,704 tokens) of Base, RFT-Curated, GRPO-Curated and iterated RFT (2026-09-20), decided after the test results were seen (Appendix G).

One difference from the registered text is not in this list. H3 as registered states "identical prompt and sampling budgets". In the experiment as run, the Curated pair is matched on prompts but not on rollouts: RFT-Curated draws 14,016 completions (73×192) and GRPO-Curated 19,200. The Easy and Mixed pairs are matched on both (§2.3), but those comparisons were added after the test results were seen.

Added after the test results were seen.

- The iterated-RFT arm and its two comparisons, registered on 2026-09-20 before any iterated-RFT sample existed; its prediction was "small positive — a few points, maybe not past seed noise" for iterated RFT over RFT-Curated.
- The GRPO–RFT comparisons on the easy and mixed prompts.
- The truncation bounds and the larger-cap re-evaluation.

The protocol also flags any run whose `test_300` truncation exceeds 5% as not a headline number as it stands. Every seed of GRPO-Mixed and GRPO-Curated exceeds it (5.3–10.3%); the truncation bounds in Appendix G are the check we report for those comparisons.

B TASK GENERATOR AND SPLITS

Operators. Following Bauer et al. (2026), a problem is an integer range $[lo, hi]$, then 1–4 filters, then 0–3 transformations, then one final operation. The answer is computed by executing the pipeline in Python and checked in tests against an independent reference implementation.

- **Filters:** even, odd, divisible-by-N, not-divisible-by-N, below-threshold, above-threshold, digit-sum-equals, contains-digit, prime, perfect-square.
- **Transformations:** add-k, multiply-by-k, square, absolute value, modulo-m, digit sum, reverse digits.
- **Final operations:** count, unique-count, zero-count, sum, product mod m, mean (rounded), median, mode, min, max, range, bitwise AND/OR/XOR.

Each operator has at least three paraphrases, and connectives depend on position ("First", "Then"/"Next", "Finally").

Our constraints.

- Every step must change the set: a pipeline is rejected if a filter or transformation leaves the multiset unchanged, if two consecutive filters are identical, if fewer than three values reach the final operation, if `mode` or `unique-count` has no repeated value, or if $|\text{answer}| > 10^9$.
- `positive` and `negative` filters are removed, since with $lo \geq 1$ they never change the set.
- Range spans fall in disjoint bands: S $[10, 50]$, M $[51, 200]$, L $[201, 1000]$.
- Generation is stratified, with equal counts per (range band $\times$ step count) cell.

Pools and splits. The main pool has 6,000 problems (bands S and M, 2–5 steps, seed 20260901); `ood_hard_200` is generated separately (band L, 6–8 steps, seed 20260902). Splits are disjoint by problem id and by pipeline structure, and were frozen before training.

Table 4: Split sizes and tier composition. `ood_hard_200` is tiered after generation but not filtered.

Split	n	easy / medium / hard	steps
`train_easy_100`	100	100 / 0 / 0	2–5
`train_mixed_100`	100	33 / 33 / 34	2–5
`train_curated`	73	26 / 33 / 14	2–5
`val_mixed_100`	100	33 / 33 / 34	2–5
`test_300`	300	100 / 100 / 100	2–5
`ood_hard_200`	200	14 / 47 / 139	6–8

Example problems (from `test_300`, verbatim):

- *Easy (5 steps), answer -9:* Start with the inclusive integer range [7, 32]. First, keep only the values that leave no remainder when divided by 7. Next, multiply each remaining number by 3. Next, take every remaining value modulo 7, using the non-negative remainder (0 to 6). Next, subtract 9 from each remaining number. Of these numbers, report the most frequent remaining value, breaking ties by taking the smallest.
- *Hard (5 steps), answer 652:* Consider the integers from 3 to 51, inclusive. First, keep only the numbers that are even. Then, keep only the numbers that are divisible by 8. Next, square each remaining number. Then, replace every remaining value x with the digits of |x| written in reverse order, with the original sign restored and leading zeros dropped. Of these numbers, report the median of the remaining numbers, taking the lower middle value when the count is even.

C ANSWER EXTRACTION

The extracted answer is the last integer, by position, found in the first of these that is present:

1. an *answer line*: a line reading `Answer:` `<integer>` once presentation-only decoration is removed (markdown headings, bullets and emphasis; code spans; `$...$`, `\boxed{}`, `\text{}` or angle-bracket wrapping of the integer; an optional "Final" and a trailing period or comma);
2. a `\boxed{<integer>}` anywhere;
3. the last integer anywhere in the completion.

Digit-group commas are removed. A completion fails extraction only if it contains no integer or reaches the token cap; a truncated completion is never scored from a trailing integer. Taking the last integer means a completion that lists several candidates is scored on the one it ends with.

Why the rule is layered. An earlier rule required an exact answer line and rejected common formats such as `**Answer: 36**`. On the 49,600-completion tiering run it scored 3,326 correct answers as wrong; 23.6% of the problems it labeled hard (678 of 2,843) were answered correctly

but in a rejected format. The layered rule reduced extraction failure on that run from 37.1% to 0.9%. Where the strict layer and the fallback both produce an answer they agree 98.4% of the time. The rule was changed after tiering and before any training run, and applied identically to every arm, control and reference model. We call the share of completions answered through layer 1 or 2 the *answer-line rate* (§3.1).

D TOKEN CAP

The cap is $\max(2048,\ \lceil 1.25 \cdot p_{99,\max} \rceil_{256})$, where $p_{99,\max}$ is the largest per-cell 99th percentile of the length of *correct* base-model completions and $\lceil \cdot \rceil_{256}$ rounds up to a multiple of 256. It was computed once, before the splits existed, from 500 random pool problems $\times$ 8 samples at T=1.0 (provisional cap 4,096; 998 correct completions). The binding cell is M3: $1.25 \times 3{,}477.9 = 4{,}347.4 \rightarrow 4{,}352$. No correct completion in the calibration run reached the provisional cap. Rescoring the same completions under the final extraction rule gives $p_{99,\max} = 3{,}161.7$ and a cap of 4,096; we kept 4,352 as the more conservative bound.

Table 5: Per-cell length statistics of correct base-model completions (tokens) used to set the cap. Cell = range band and total steps.

Cell	n correct	median	p_{99}	over 2,048 (%)
S2	139	261	1,183	0.0
S3	172	346	1,690	1.2
S4	152	463	1,431	0.0
S5	133	552	2,114	1.5
M2	95	400	2,675	4.2
M3	103	392	3,478	2.9
M4	122	531	1,543	0.8
M5	82	617	2,202	2.4

E HYPERPARAMETERS

Table 6 lists the training settings and Table 7 the RFT sweep.

Other package versions: Python 3.11.16, PyTorch 2.13.0, Transformers 5.16.1, PEFT 0.20.0.

F PER-SEED RESULTS

Table 8 lists every seed's greedy accuracy and truncation rate.

G TRUNCATION

Bounds. A truncated completion is scored wrong, and inspection of the greedy outputs found that truncated completions contain no final answer before the cut. To bound how much truncation could move each comparison, we rescore each run at two extremes: every truncated completion wrong (as reported) or every truncated completion right. The lower bound of a difference A $-$ B credits B's truncated completions and not A's; the upper bound does the reverse. These bounds were defined after the test results were seen.

Larger cap (exploratory). We re-generated greedy completions at twice the cap (8,704 tokens) for Base, RFT-Curated, iterated RFT and GRPO-Curated. The models were still trained at 4,352. At the larger cap GRPO-Curated leads RFT-Curated by 15.0 pp on test [+11.1, +19.1] and 11.5 pp on OOD [+6.0, +17.0], and leads iterated RFT by 11.9 and 10.7 pp. GRPO-Curated is still cut on 11–31% of OOD problems at the larger cap.

Table 6: Training settings. Values marked * are library defaults recorded from the resolved configuration.

Shared	
Model	`Qwen/Qwen3-4B-Base`, bf16
LoRA	rank 64, α=16, dropout 0; q, k, v, o, gate, up, down
Optimizer	AdamW, weight decay 0, gradient clip 1.0
Schedule	cosine, 10% warmup
Sampling	T=1.0, top-p 1.0, cap 4,352 completion / 4,096 prompt tokens
Seeds	1, 2, 3 (LoRA init, data order, sampling)
RFT	
Draw	192 samples per prompt from the base model, once (draw seed 1)
Acceptance	verified correct; exact duplicates removed; no per-prompt cap
Loss	completion-only NLL; end-of-sequence token trained
Batch	16 (micro-batch 4 $\times$ 4 accumulation)
Sweep	lr $\{10^{-5}, 5{\times}10^{-5}, 10^{-4}\}$ $\times$ epochs $\{2, 4, 8\}$, seed 1
Selected	Easy $5{\times}10^{-5}$/8; Mixed $5{\times}10^{-5}$/4; Curated 10^{-5}/4
GRPO (TRL `GRPOTrainer` 1.13.0, colocated vLLM 0.28.0)	
Steps	300; 8 prompts $\times$ 8 completions per step
Learning rate	$5{\times}10^{-5}$
Objective	group-normalized advantage; KL coefficient 0; clip 0.2; `loss_type=dapo`
Updates	one per generation batch (`num_iterations=1`); 64 completions per step
Batch	micro-batch 4 $\times$ 16 accumulation
Truncated rollouts	reward 0, kept in the loss (`mask_truncated_completions=False`)
Optimizer	`adamw_torch_fused`, $\beta = (0.9, 0.999)^*$, $\epsilon = 10^{-8}$ *
Iterated RFT	
Rounds	3 $\times$ 64 samples per prompt on the 73 curated prompts (14,016)
Sampling	round 1 from the base draw; rounds 2–3 from the current policy
Training	continue previous adapter; fresh optimizer and schedule; lr 10^{-5}, 4 epochs
Controls (GRPO recipe on `train_mixed_100`, seed 1)	
C1	reward $\sim$ Bernoulli(0.5), seeded per (seed, problem, step, sample)
C2	reward 1 iff an explicit final answer (layer 1 or 2) and not truncated

Table 7: RFT sweep: `val_mixed_100` greedy accuracy (%, $n = 100$, seed 1) for each learning rate and epoch count. Bold: selected (ties broken by fewer epochs, then lower learning rate).

	lr 10^{-5}			lr $5{\times}10^{-5}$			lr 10^{-4}		
Epochs	2	4	8	2	4	8	2	4	8
RFT-Easy	59	64	61	63	62	**67**	61	64	63
RFT-Mixed	62	62	66	65	**67**	63	62	67	57
RFT-Curated	62	**66**	65	63	64	60	64	64	58

Table 8: Greedy accuracy (%) by seed, with the share of completions truncated at the cap in parentheses. Mean $\pm$ sd is in Table 2.

	`test_300`			`ood_hard_200`		
Arm	seed 1	seed 2	seed 3	seed 1	seed 2	seed 3
RFT-Easy	66.3 (2.3)	64.7 (3.3)	66.7 (1.7)	25.0 (14.0)	26.0 (9.5)	25.5 (8.0)
RFT-Mixed	70.7 (2.7)	68.7 (1.7)	69.3 (1.7)	28.0 (12.0)	28.0 (11.0)	26.0 (13.5)
RFT-Curated	67.3 (2.7)	65.7 (3.3)	67.7 (3.3)	30.0 (14.0)	29.0 (15.5)	26.5 (15.0)
GRPO-Easy	77.7 (4.3)	70.0 (3.3)	75.3 (1.3)	34.5 (38.0)	30.5 (21.0)	36.5 (19.5)
GRPO-Mixed	78.7 (6.7)	79.3 (10.3)	78.3 (9.0)	29.5 (46.5)	29.5 (54.5)	33.5 (41.5)
GRPO-Curated	79.3 (5.3)	76.0 (9.3)	78.0 (8.0)	36.0 (19.5)	29.0 (45.0)	29.5 (40.0)
IterRFT-Curated	69.7 (1.3)	70.0 (1.7)	70.7 (1.7)	28.0 (14.5)	27.0 (16.0)	31.5 (14.5)

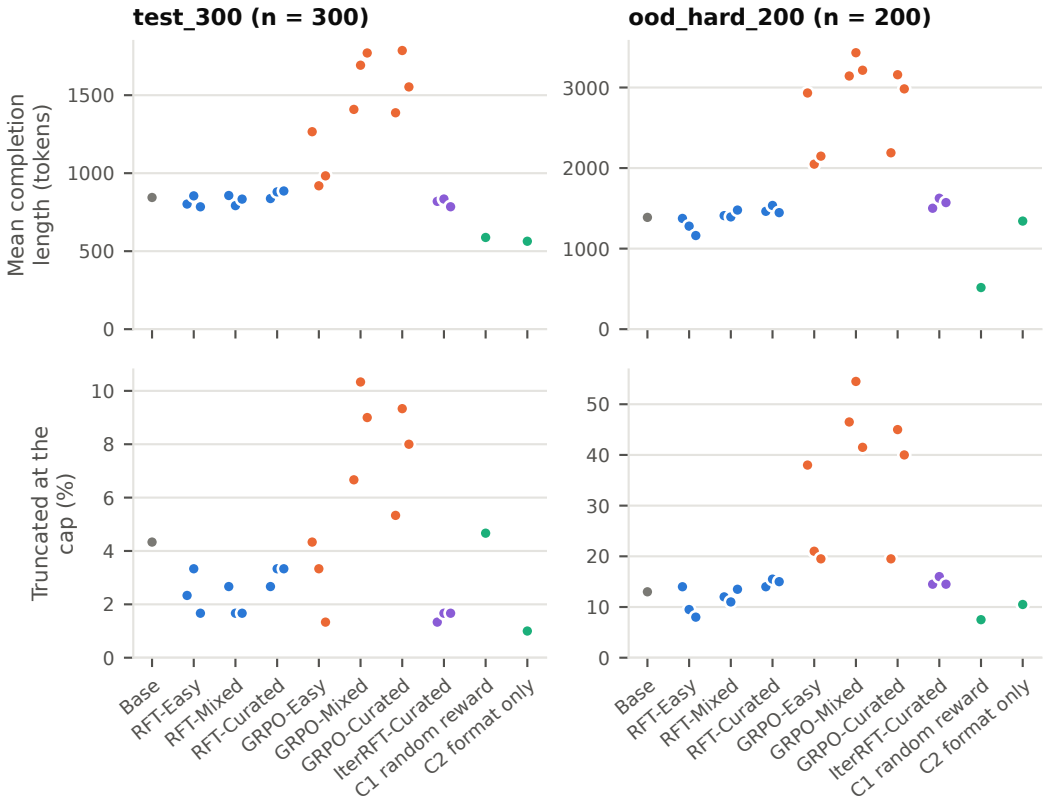


Figure 3: Mean completion length (top) and share of completions truncated at the 4,352-token cap (bottom) under greedy decoding. Each dot is one seed (three per arm; one for Base, C1 and C2). Colors: gray Base, blue RFT, orange GRPO, purple iterated RFT, green controls. GRPO-Mixed and GRPO-Curated write longer completions and are truncated more often than RFT, especially on the out-of-distribution set.

Table 9: Truncation bounds on `test_300` (pp): the observed difference and its range when truncated completions are counted against A (lower) or against B (upper). On `ood_hard_200` every interval includes zero.

Comparison (A − B)	observed	lower	upper
GRPO-Curated − RFT-Curated	+10.9	+7.8	+18.4
GRPO-Easy − RFT-Easy	+8.4	+6.0	+11.4
GRPO-Mixed − RFT-Mixed	+9.2	+7.2	+17.9
RFT-Curated − RFT-Mixed	−2.7	−4.7	+0.4
RFT-Mixed − RFT-Easy	+3.7	+1.2	+5.7
GRPO-Mixed − GRPO-Easy	+4.4	+1.4	+13.1
IterRFT-Curated − RFT-Curated	+3.2	+0.1	+4.8
GRPO-Curated − IterRFT-Curated	+7.7	+6.1	+15.2

Reproducibility across machines. Greedy decoding with vLLM was byte-identical when a model was re-evaluated on the same machine (1,396 of 1,396 completions), but not across machines: 6–10% of problems changed outcome, moving accuracy by up to 2.3 pp on test and 5.0 pp on OOD. Runs were evaluated on several machines, so this variation is not contained in the seed spread. It is small next to the GRPO–RFT differences but comparable to the curation, mixed-data (RFT) and iterated-RFT effects.

H PASS@k AND SAMPLED ACCURACY

Figure 4 shows pass@k for every arm, and Table 10 reports sampled accuracy.

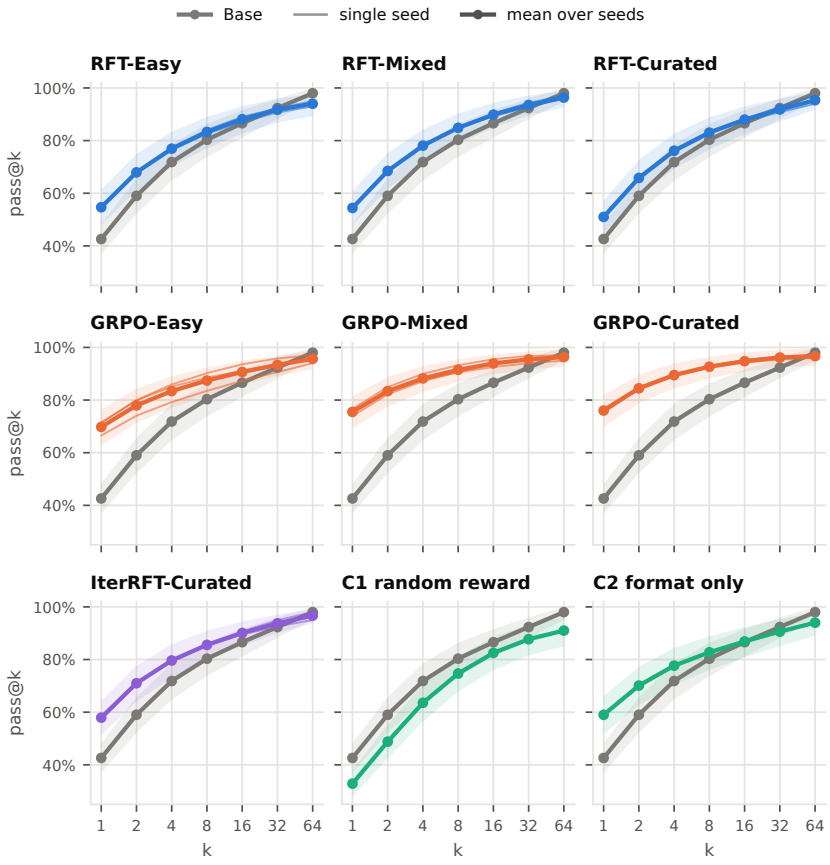


Figure 4: pass@k on the first 100 `test_300` problems, from 64 samples per problem at T=1.0 (unbiased estimator). Thin lines are seeds; thick lines are the mean over seeds; shaded bands are 95% bootstrap CIs over the 100 problems, for Base (gray) as well. Arms are colored: RFT blue, GRPO orange, iterated RFT purple, controls green. C1 and C2 are single seeds (one curve, no seed lines). GRPO lifts pass@k at small k and meets Base, within the bands, at $k = 64$.

Table 10: Sampled accuracy (%): mean@8 at T=1.0 on `test_300` and `ood_hard_200`, and pass@8 / pass@64 on the first 100 test problems. Mean over seeds; controls and Base are single runs.

Arm	test mean@8	OOD mean@8	pass@8	pass@64
Base	42.5	17.3	80.3	98.0
RFT-Easy	53.5	24.3	83.3	94.0
RFT-Mixed	54.4	23.5	84.9	96.3
RFT-Curated	51.2	21.6	83.0	95.3
GRPO-Easy	70.1	33.0	87.4	95.7
GRPO-Mixed	77.1	31.1	91.5	96.3
GRPO-Curated	76.3	34.8	92.7	96.7
IterRFT-Curated	58.5	25.0	85.6	96.7
C1 random reward	33.6	14.0	74.7	91.0
C2 format reward	60.3	26.4	82.7	94.0

I ADDITIONAL RESULTS

GSM8K transfer. Greedy accuracy on the first 500 GSM8K test problems, same prompt and cap: Base 88.0%; RFT-Easy 91.9%, RFT-Mixed 90.9%, RFT-Curated 91.5%; GRPO-Easy 93.3%, GRPO-Mixed 92.7%, GRPO-Curated 92.9%; iterated RFT 92.1%; C1 85.0%; C2 93.2%. Truncation is at most 1.0% for every run. We computed no comparisons on this set; reference models were not evaluated on it.

Reference models. Greedy `test_300` / `ood_hard_200` accuracy, same prompt inside each model's chat template: Qwen3-4B (non-thinking) 74.7 / 45.5%; Qwen2.5-7B-Instruct 65.3 / 28.0%; Llama-3.1-8B-Instruct 37.3 / 9.0% (11.3% of its test completions truncated).

J COMPUTE

All runs used one NVIDIA H100 PCIe 80 GB GPU per job. Table 11 gives metered GPU-hours per arm, including the RFT sweep and all evaluations. Arms and controls total 212.1 GPU-hours; with base and reference evaluations, tiering, the shared RFT draw, the exploratory re-evaluation, and discarded runs (RFT before the end-of-sequence fix, and a failed GRPO attempt), the metered total is 252.7 GPU-hours.

Table 11: Metered GPU-hours (H100) per arm. "Sweep" is the eight non-selected RFT configurations.

Arm	training (selected)	sweep	evaluation	total
RFT-Easy	21.3	30.1	1.8	53.1
RFT-Mixed	7.0	22.1	1.9	31.0
RFT-Curated	5.9	18.5	1.9	26.3
GRPO-Easy	15.5	–	3.0	18.5
GRPO-Mixed	27.1	–	5.0	32.0
GRPO-Curated	25.1	–	4.4	29.4
IterRFT-Curated	8.0	–	1.9	9.8
C1 random reward	5.9	–	0.5	6.4
C2 format reward	5.0	–	0.5	5.5
Total				212.1